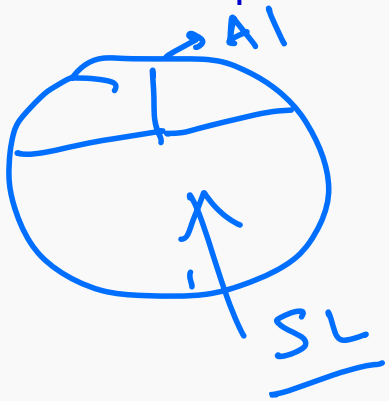
Foundations for AI

Dr. Rishikesh Yadav & Vedant Vibhor

May 2026

School of Mathematical and Statistical Sciences
Indian Institute of Technology Mandi

Table of Contents



Introduction

Main Objectives

- Build solid foundations with **NumPy**: **arrays**, **vectorization**, and **linear algebra** for AI computations.

Main Objectives

- Build solid foundations with **NumPy**: **arrays**, **vectorization**, and **linear algebra** for AI computations.
- Use **Pandas** to turn raw data into clean tables: **loading**, **cleaning**, and **feature selection**.

Main Objectives

- Build solid foundations with **NumPy**: **arrays**, **vectorization**, and **linear algebra** for AI computations.
- Use **Pandas** to turn raw data into clean tables: **loading**, **cleaning**, and **feature selection**.
- Practice **EDA** to spot **patterns**, **correlations**, and **outliers** before modeling.

Main Objectives

- Build solid foundations with **NumPy**: **arrays**, **vectorization**, and **linear algebra** for AI computations.
- Use **Pandas** to turn raw data into clean tables: **loading**, **cleaning**, and **feature selection**.
- Practice **EDA** to spot **patterns**, **correlations**, and **outliers** before modeling.
- Understand **supervised learning**: mapping X (features) and y (targets) to **regression** and **classification**.

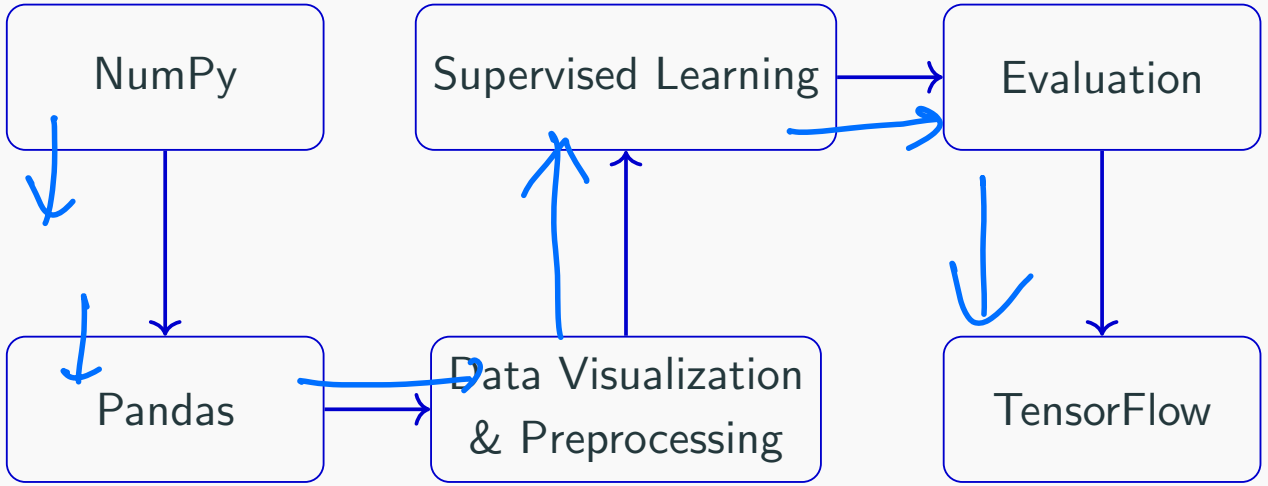
Main Objectives

- Build solid foundations with **NumPy**: **arrays**, **vectorization**, and **linear algebra** for AI computations.
- Use **Pandas** to turn raw data into clean tables: **loading**, **cleaning**, and **feature selection**.
- Practice **EDA** to spot **patterns**, **correlations**, and **outliers** before modeling.
- Understand **supervised learning**: mapping X (features) and y (targets) to **regression** and **classification**.
- Evaluate models using appropriate **metrics** and build intuition for **overfitting vs. generalization**.

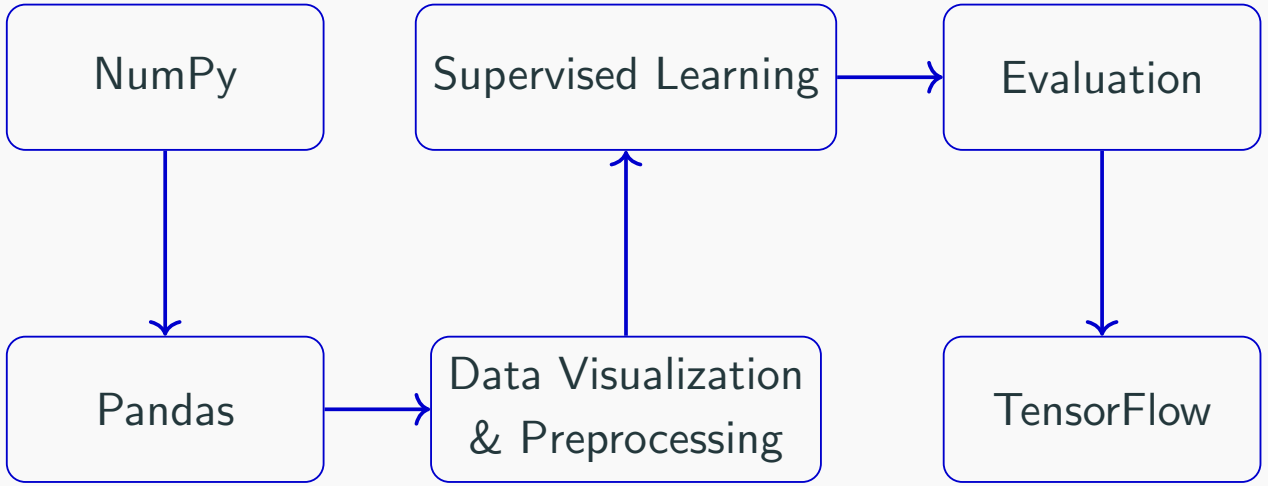
Main Objectives

- Build solid foundations with **NumPy**: **arrays**, **vectorization**, and **linear algebra** for AI computations.
- Use **Pandas** to turn raw data into clean tables: **loading**, **cleaning**, and **feature selection**.
- Practice **EDA** to spot **patterns**, **correlations**, and **outliers** before modeling.
- Understand **supervised learning**: mapping X (features) and y (targets) to **regression** and **classification**.
- Evaluate models using appropriate **metrics** and build intuition for **overfitting vs. generalization**.
- Get started with **TensorFlow/Keras** to define, train, and run a simple **neural network**.

Roadmap

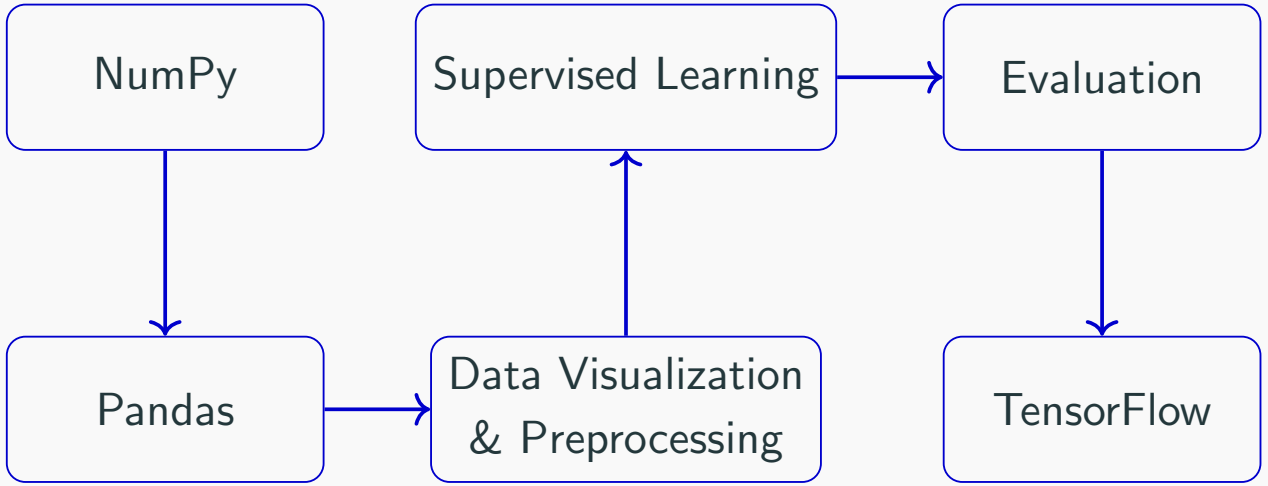

NumPy
Supervised Learning
Evaluation
Pandas
Data Visualization
& Preprocessing
TensorFlow

Roadmap



- **NumPy** → fast numerical ops (arrays, broadcasting)
- **Pandas** → real-world data handling (cleaning, missing values)
- **EDA** → plots & correlations to guide choices
- **Supervised learning** → baseline models (regression/classification)
- **Evaluation** → pick the right metric (MAE/RMSE, accuracy/F1)
- **TensorFlow** → train a small neural network

Roadmap



- **NumPy** → fast numerical ops (arrays, broadcasting)
- **Pandas** → real-world data handling (cleaning, missing values)
- **EDA** → plots & correlations to guide choices
- **Supervised learning** → baseline models (regression/classification)
- **Evaluation** → pick the right metric (MAE/RMSE, accuracy/F1)
- **TensorFlow** → train a small neural network

Theory (∼2.5–3 hrs) + Hands-on Lab (Separate Session)

Numpy: Foundation of Numerical Computing

Why NumPy for AI?

- **Common array language** across **Pandas**, **scikit-learn**, **PyTorch**, and **TensorFlow**.
- **Speed matters**: vectorized ops run in optimized **C/Fortran** (often via **BLAS/LAPACK**) instead of slow Python loops.
- **Practical toolkit**: broadcasting, random sampling, and linear algebra that show up everywhere in ML/DL.

Why NumPy for AI?

- **Common array language** across **Pandas**, **scikit-learn**, **PyTorch**, and **TensorFlow**.
- **Speed matters**: vectorized ops run in optimized **C/Fortran** (often via **BLAS/LAPACK**) instead of slow Python loops.
- **Practical toolkit**: broadcasting, random sampling, and linear algebra that show up everywhere in ML/DL.

A 30-second history

Python first had multiple array libraries (**Numeric** and **numarray**). In the early 2000s they were unified into **NumPy** (led by **Travis Oliphant**), giving Python a fast, MATLAB-like **n-D array** foundation that today's ML stack builds on.

Why NumPy for AI?

- **Common array language** across **Pandas**, **scikit-learn**, **PyTorch**, and **TensorFlow**.
- **Speed matters**: vectorized ops run in optimized **C/Fortran** (often via **BLAS/LAPACK**) instead of slow Python loops.
- **Practical toolkit**: broadcasting, random sampling, and linear algebra that show up everywhere in ML/DL.

A 30-second history

Python first had multiple array libraries (**Numeric** and **numarray**). In the early 2000s they were unified into **NumPy** (led by **Travis Oliphant**), giving Python a fast, MATLAB-like **n-D array** foundation that today's ML stack builds on.

Core idea: Modern AI = **Data** + **Matrix Operations** + **Optimization**

Python Lists vs NumPy Arrays

Key idea: Python lists are **general containers**; NumPy arrays are **numerical vectors/matrices** designed for fast math.

Python Lists vs NumPy Arrays

Key idea: Python lists are **general containers**; NumPy arrays are **numerical vectors/matrices** designed for fast math.

Python List (container semantics)

```
a = [1, 2, 3]
b = [4, 5, 6]
a + b          # Concatenation
```

Output:

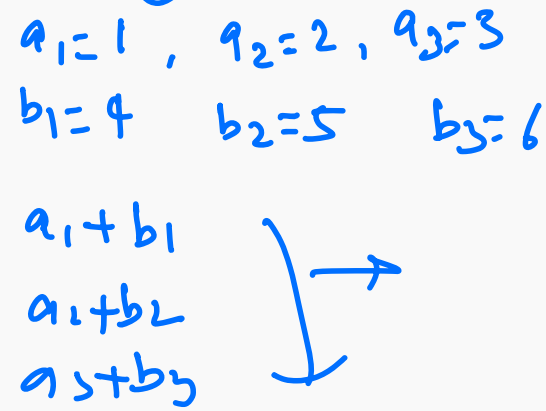
```
[1, 2, 3, 4, 5, 6]
```

NumPy Array (numeric semantics)

```
import numpy as np
a = np.array([1,2,3])
b = np.array([4,5,6])
a + b          # Element-wise addition
```

Output:

```
[5 7 9]
```



Python Lists vs NumPy Arrays

Key idea: Python lists are **general containers**; NumPy arrays are **numerical vectors/matrices** designed for fast math.

Python List (container semantics)

```
a = [1, 2, 3]
b = [4, 5, 6]
a + b          # Concatenation
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

NumPy Array (numeric semantics)

```
import numpy as np
a = np.array([1,2,3])
b = np.array([4,5,6])
a + b          # Element-wise addition
```

Output:

```
[5 7 9]
```

Why AI cares: arrays enable **vectorization** (fast C/BLAS under the hood) and make math look like the equations.

Understanding Arrays

```
import numpy as np                      # use NumPy

a = np.array([1, 2, 3])                 # create from a Python list
zeros = np.zeros((2, 3))                # initialize with zeros (shape 2x3)
ones = np.ones((2, 2))                  # initialize with ones (shape 2x2)
eye = np.eye(3)                         # identity matrix (3x3)
seq = np.arange(0, 10, 2)               # integer sequence: start, stop, step
lin = np.linspace(0, 1, 5)              # evenly spaced values between 0 and 1
rand = np.random.rand(2, 3)             # random samples in [0,1), shape 2x3
```

Understanding Arrays

```python
import numpy as np                      # use NumPy

a = np.array([1, 2, 3])                 # create from a Python list
zeros = np.zeros((2, 3))                # initialize with zeros (shape 2x3)
ones = np.ones((2, 2))                  # initialize with ones (shape 2x2)
eye = np.eye(3)                         # identity matrix (3x3)
seq = np.arange(0, 10, 2)               # integer sequence: start, stop, step
lin = np.linspace(0, 1, 5)              # evenly spaced values between 0 and 1
rand = np.random.rand(2, 3)             # random samples in [0,1), shape 2x3
```

Outputs (examples)

```
a     → [1 2 3]
zeros → [[0. 0. 0.]
         [0. 0. 0.]]
eye   → [[1. 0. 0.]
         [0. 1. 0.]
         [0. 0. 1.]]
```

Understanding Arrays

```python
import numpy as np                          # use NumPy

a = np.array([1, 2, 3])                     # create from a Python list
zeros = np.zeros((2, 3))                    # initialize with zeros (shape 2x3)
ones = np.ones((2, 2))                      # initialize with ones (shape 2x2)
eye = np.eye(3)                             # identity matrix (3x3)
seq = np.arange(0, 10, 2)                   # integer sequence: start, stop, step
lin = np.linspace(0, 1, 5)                  # evenly spaced values between 0 and 1
rand = np.random.rand(2, 3)                 # random samples in [0,1), shape 2x3
```

Outputs (examples)

```
a     → [1 2 3]
zeros → [[0. 0. 0.]
         [0. 0. 0.]]
eye   → [[1. 0. 0.]
         [0. 1. 0.]
         [0. 0. 1.]]
```

Common AI usage

- **Initialize** weights/biases
- **Create** synthetic batches
- Build **feature matrices** X

Understanding Array Dimensions

```
x = np.array([1,2,3])
A = np.array([[1,2], [3,4]])
```

Understanding Array Dimensions

```
x = np.array([1,2,3])
A = np.array([[1,2], [3,4]])
```

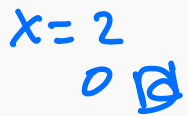
Property	x	A	B	C
shape	(3,)	(2,2)	(3,2,4)	(2,3,3,4)
ndim	1	2	3	
size	3	4		
dtype	int	int		

Understanding Array Dimensions

```
x = np.array([1,2,3])
A = np.array([[1,2], [3,4]])
```

Property	x	A
shape	(3,)	(2,2)
ndim	1	2
size	3	4
dtype	int	int

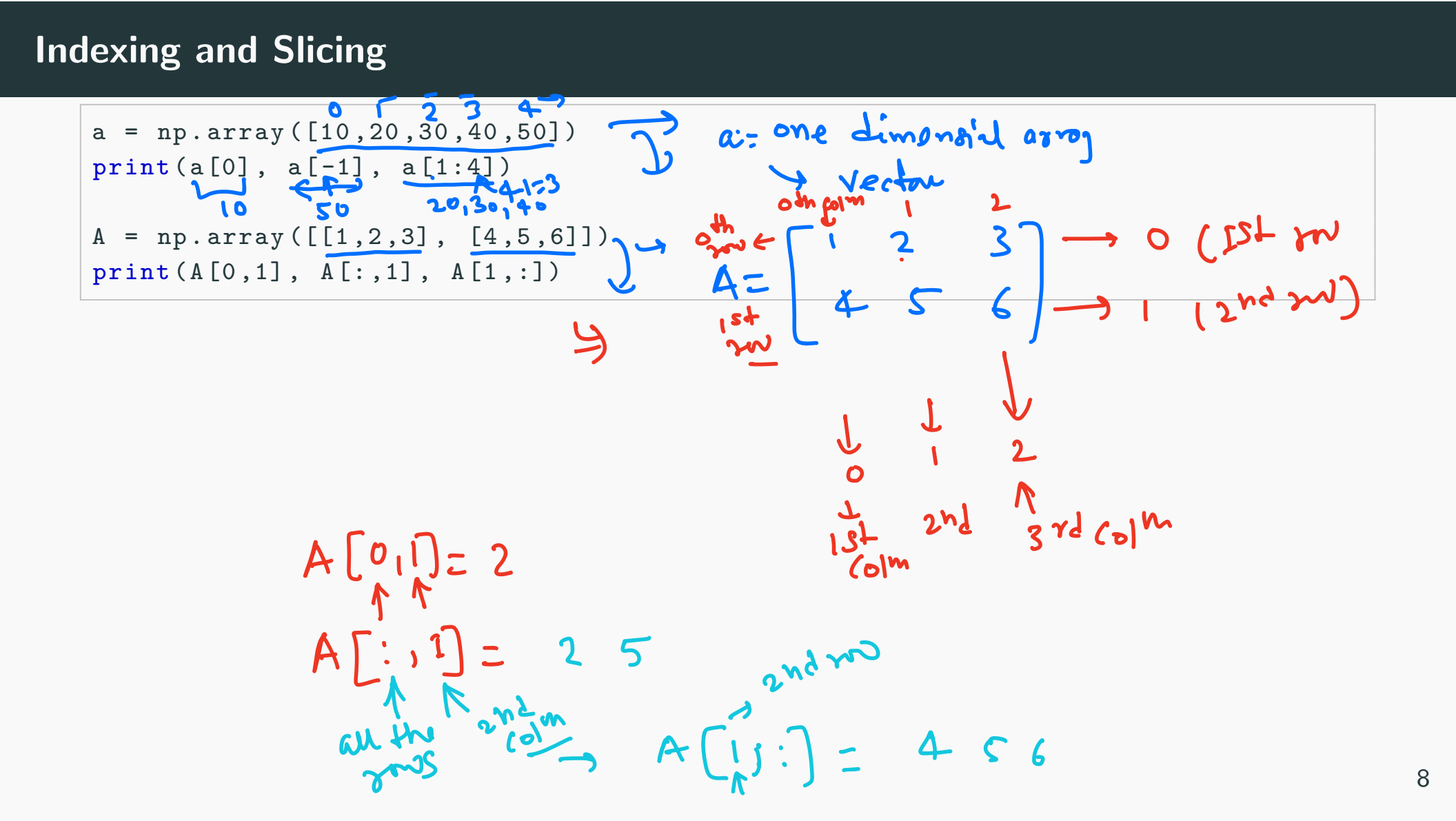
Terminology



- Scalar: single number
- Vector: 1D array
- Matrix: 2D array
- Tensor: n-dimensional array

Indexing and Slicing

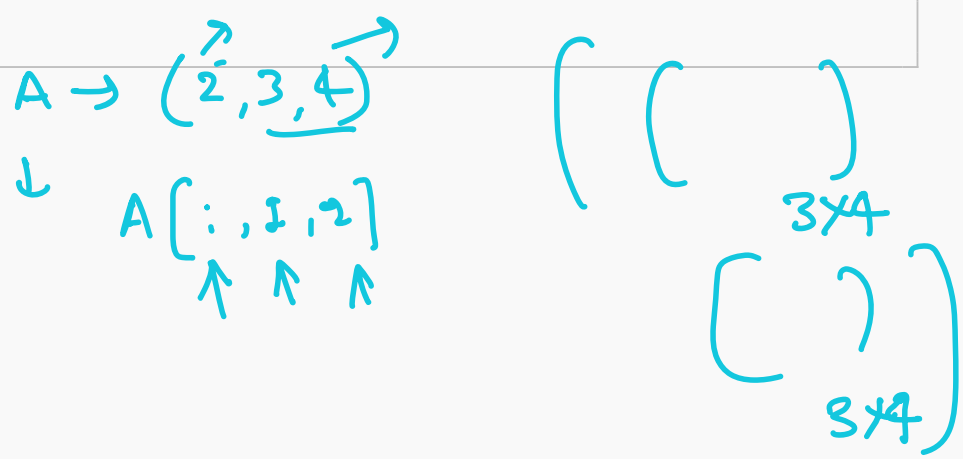
```
a = np.array([10,20,30,40,50])
print(a[0], a[-1], a[1:4])
A = np.array([[1,2,3], [4,5,6]])
print(A[0,1], A[:,1], A[1,:])
```



Indexing and Slicing

```
a = np.array([10,20,30,40,50])
print(a[0], a[-1], a[1:4])

A = np.array([[1,2,3], [4,5,6]])
print(A[0,1], A[:,1], A[1,:])
```



Output:

```
10 50 [20 30 40]
2 [2 5] [4 5 6]
```

Indexing and Slicing

```
a = np.array([10,20,30,40,50])
print(a[0], a[-1], a[1:4])

A = np.array([[1,2,3], [4,5,6]])
print(A[0,1], A[:,1], A[1,:])
```

Output:

```
10 50 [20 30 40]
2 [2 5] [4 5 6]
```

AI Relevance

- Selecting features
- Train-test splitting
- Batch extraction

Vectorized Operations

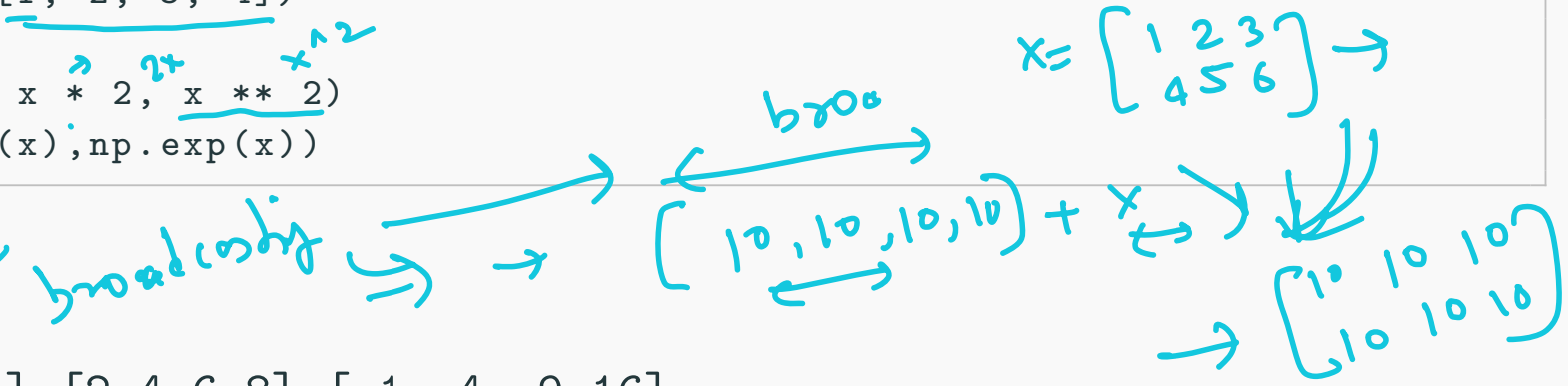
```
x = np.array([1, 2, 3, 4])

print(x + 10, x * 2, x ** 2)
print(np.sqrt(x),np.exp(x))
```

Vectorized Operations

```
x = np.array([1, 2, 3, 4])

print(x + 10, x * 2, x ** 2)
print(np.sqrt(x),np.exp(x))
```



Output:

```
[11 12 13 14] [2 4 6 8] [ 1  4  9 16]
[1.    1.414 1.732 2.   ] [ 2.718  7.389 20.085 54.598]
```

Vectorized Operations

```
x = np.array([1, 2, 3, 4])

print(x + 10, x * 2, x ** 2)
print(np.sqrt(x),np.exp(x))
```

Output:

```
[11 12 13 14] [2 4 6 8] [ 1  4  9 16]
[1.    1.414 1.732 2.   ] [ 2.718  7.389 20.085 54.598]
```

Vectorization

- Apply operations to entire arrays.
- Avoid explicit Python loops.
- Essential for efficient AI computations.

Broadcasting

```
A = np.array([[1,2,3], [4,5,6]])
b = np.array([10,20,30])
print(A + b)
```

$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ → shape (2,3)

$b = (10 \quad 20 \quad 30)$ → shape (3,)

$A + b = \begin{bmatrix} 10 & 20 & 30 \\ 10 & 20 & 30 \end{bmatrix} + \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$

$= \begin{bmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \end{bmatrix}$

Broadcasting

```
A = np.array([[1,2,3], [4,5,6]])
b = np.array([10,20,30])

print(A + b)
```

Output:

$$\begin{bmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \end{bmatrix}$$

Broadcasting

```
A = np.array([[1,2,3], [4,5,6]])
b = np.array([10,20,30])

print(A + b)
```

Output:

$$\begin{bmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \end{bmatrix}$$

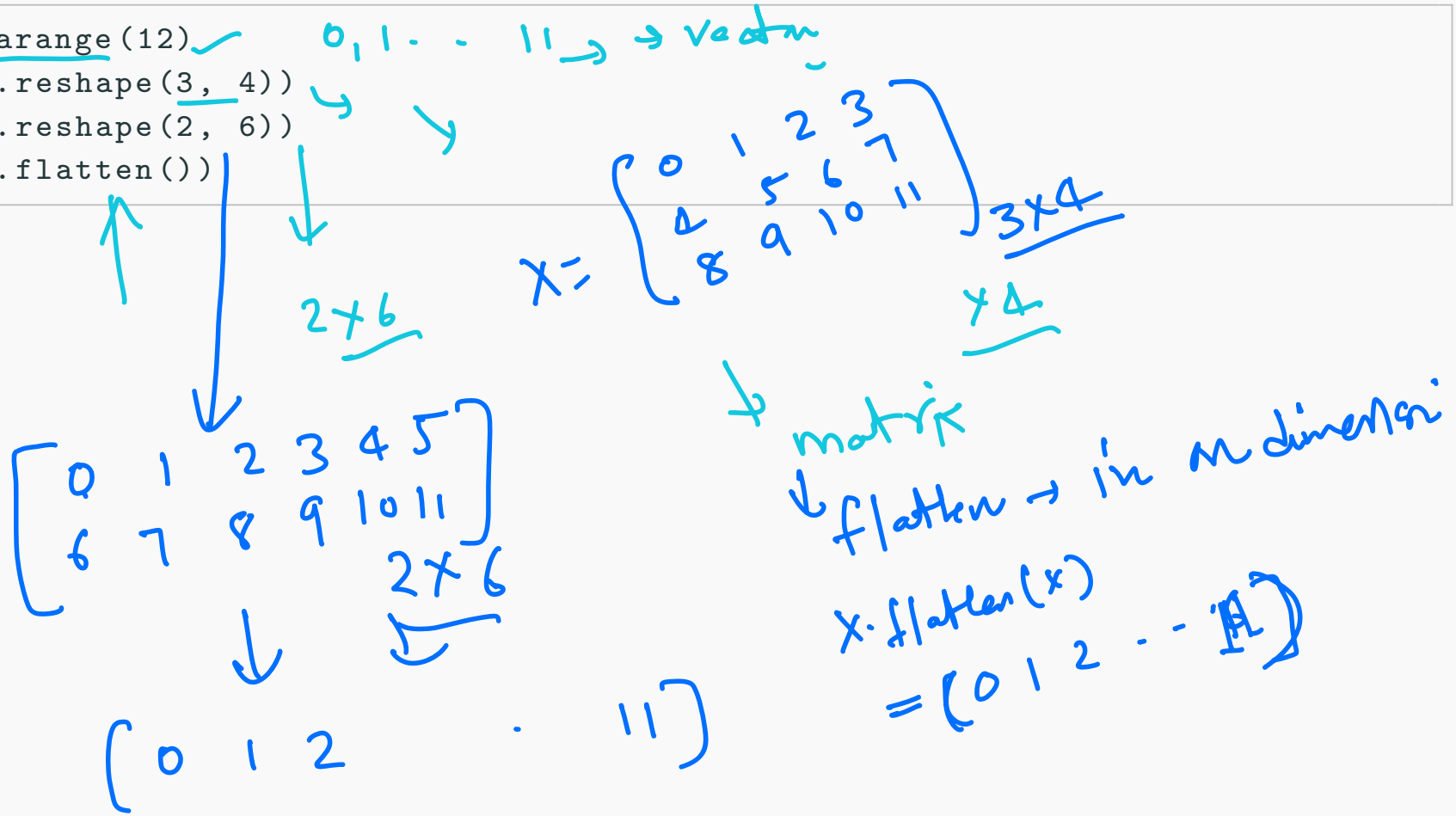
Broadcasting Rule

- Smaller arrays automatically expand.
- Eliminates unnecessary copying.

AI Example: Adding bias terms in neural networks.

Reshaping Arrays

```
x = np.arange(12)
print(x.reshape(3, 4))
print(x.reshape(2, 6))
print(x.flatten())
```



Reshaping Arrays

```
x = np.arange(12)
print(x.reshape(3, 4))
print(x.reshape(2, 6))
print(x.flatten())
```

Output (partial):

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

3 X 4

Reshaping Arrays

```
x = np.arange(12)
print(x.reshape(3, 4))
print(x.reshape(2, 6))
print(x.flatten())
```

Output (partial):

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

Why Important?

- Images need reshaping.
- Data preparation for ML models.
- Mini-batch organization.

Aggregation Functions

```
x = np.array([2,4,6,8])

print(np.sum(x))
print(np.mean(x), np.std(x))
print(np.min(x), np.max(x))
```

Output:

```
20
5.0 2.236...
2 8
```

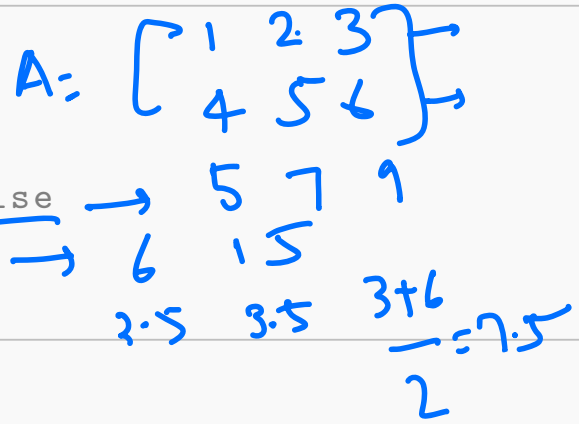
Used For

- Data analysis
- Feature engineering
- Model evaluation metrics

Working Along Axes

```
A = np.array([[1,2,3],
              [4,5,6]])

print(A.sum(axis=0))    # column-wise
print(A.sum(axis=1))    # row-wise
print(A.mean(axis=0))
```



Output:

```
[5 7 9]
[ 6 15]
[2.5 3.5 4.5]
```

AI Example

- Compute feature means.
- Normalize datasets.

Linear Algebra Essentials

```
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])

print(A @ B)              # Matrix multiplication
print(A.T)                # Transpose
print(np.linalg.det(A))
```

Output:

```
[[19 22]
 [43 50]]
[[1 3]
 [2 4]]
-2.0
```

Linear Algebra Essentials

```
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])

print(A @ B)             # Matrix multiplication
print(A.T)               # Transpose
print(np.linalg.det(A))
```

Output:

```
[[19 22]
 [43 50]]
[[1 3]
 [2 4]]
-2.0
```

Really essential for neural networks, PCA, optimization.

NumPy in Action: Linear Model

$$y = w_1 x_1 + w_2 x_2 + b$$

```
X = np.array([[2,3], [4,5], [6,7]])
w = np.array([0.5, 1.2])
b = 2

pred = X @ w + b
print(pred)
```

Output:

```
[ 6.6  10.0 13.4]
```

NumPy in Action: Linear Model

$$y = w_1 x_1 + w_2 x_2 + b$$

```
X = np.array([[2,3], [4,5], [6,7]])
w = np.array([0.5, 1.2])
b = 2

pred = X @ w + b
print(pred)
```

Output:

```
[ 6.6  10.0 13.4]
```

This matrix operation is the foundation of neural network forward passes.

Random Numbers for AI

```
np.random.seed(42)

print(np.random.rand(3))
print(np.random.randn(3))
print(np.random.randint(0, 10, 5))
```

Output (example):

```
[0.3745 0.9507 0.7320]
[-1.1119  0.3189  0.2790]
[7 2 5 4 1]
```

Random Numbers for AI

```
np.random.seed(42)

print(np.random.rand(3))
print(np.random.randn(3))
print(np.random.randint(0, 10, 5))
```

Output (example):

```
[0.3745 0.9507 0.7320]
[-1.1119  0.3189  0.2790]
[7 2 5 4 1]
```

Applications: Weight initialization, data augmentation, stochastic optimization.

Mini Exercise (1/2)

Given:

```
X = np.array([[1,2],
              [3,4],
              [5,6]])
```

$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$ 3x2

Find:

1. Shape of X

Mini Exercise (1/2)

Given:

```
X = np.array([[1,2],
              [3,4],
              [5,6]])
```

Find:

1. Shape of X

```
X.shape
```

```
(3, 2)
```

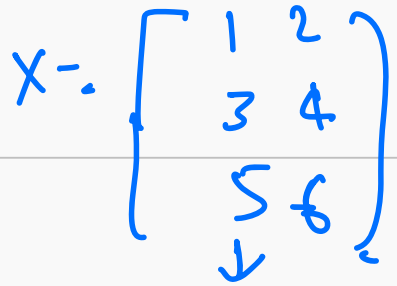
2. Mean of each column

X.mean(

Mini Exercise (1/2)

Given:

```
X = np.array([[1,2],
              [3,4],
              [5,6]])
```



Find:

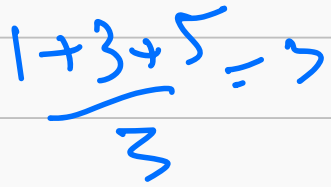
1. Shape of X

```
X.shape
```

```
(3, 2)
```

2. Mean of each column

```
X.mean(axis=0)
```



```
[3. 4.]
```

Mini Exercise (2/2)

3. Sum of each row

```
X.sum(axis=1)
```

Mini Exercise (2/2)

3. Sum of each row

```
X.sum(axis=1)
```

```
[ 3  7 11]
```

4. X multiplied by 10

```
X*10
```

Mini Exercise (2/2)

3. Sum of each row

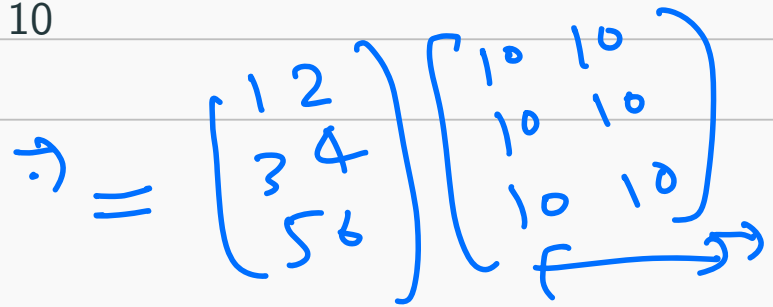
```
X.sum(axis=1)
```

```
[ 3  7 11]
```

4. X multiplied by 10

```
X * 10
```

```
[[10 20]
 [30 40]
 [50 60]]
```



5. X transpose

X.T

Mini Exercise (2/2)

3. Sum of each row

```
X.sum(axis=1)
```

```
[ 3  7 11]
```

4. X multiplied by 10

```
X * 10
```

```
[[10 20]
 [30 40]
 [50 60]]
```

5. X transpose

```
X.T
```

```
[[1 3 5]
 [2 4 6]]
```

Key Takeaways

- **ndarray** is NumPy's main object: a fast, contiguous **n-dimensional array** used to store vectors, matrices, and tensors.

Key Takeaways

- **ndarray** is NumPy's main object: a fast, contiguous **n-dimensional array** used to store vectors, matrices, and tensors.
- Always check **shape** and **ndim** before operations; most bugs in ML code are **dimension/axis** mistakes.

Key Takeaways

- **ndarray** is NumPy's main object: a fast, contiguous **n-dimensional array** used to store vectors, matrices, and tensors.
- Always check **shape** and **ndim** before operations; most bugs in ML code are **dimension/axis** mistakes.
- **Vectorization** (operate on whole arrays) replaces Python loops and is the reason NumPy code is **fast**.

Key Takeaways

- **ndarray** is NumPy's main object: a fast, contiguous **n-dimensional array** used to store vectors, matrices, and tensors.
- Always check **shape** and **ndim** before operations; most bugs in ML code are **dimension/axis** mistakes.
- **Vectorization** (operate on whole arrays) replaces Python loops and is the reason NumPy code is **fast**.
- **Broadcasting** lets different shapes work together (e.g., add a bias vector to every row of a batch) without manual copying.

Key Takeaways

- **ndarray** is NumPy's main object: a fast, contiguous **n-dimensional array** used to store vectors, matrices, and tensors.
- Always check **shape** and **ndim** before operations; most bugs in ML code are **dimension/axis** mistakes.
- **Vectorization** (operate on whole arrays) replaces Python loops and is the reason NumPy code is **fast**.
- **Broadcasting** lets different shapes work together (e.g., add a bias vector to every row of a batch) without manual copying.
- Learn the "daily tools": **indexing/slicing**, **reshape/flatten**, **axis** reductions (sum/mean), and **matrix multiplication** (`@`).

Key Takeaways

- **ndarray** is NumPy's main object: a fast, contiguous **n-dimensional array** used to store vectors, matrices, and tensors.
- Always check **shape** and **ndim** before operations; most bugs in ML code are **dimension/axis** mistakes.
- **Vectorization** (operate on whole arrays) replaces Python loops and is the reason NumPy code is **fast**.
- **Broadcasting** lets different shapes work together (e.g., add a bias vector to every row of a batch) without manual copying.
- Learn the "daily tools": **indexing/slicing**, **reshape/flatten**, **axis** reductions (sum/mean), and **matrix multiplication** (`@`).
- Strong NumPy fundamentals make **Pandas**, **Scikit-Learn**, and **TensorFlow/PyTorch** much easier.

Key Takeaways

- **ndarray** is NumPy's main object: a fast, contiguous **n-dimensional array** used to store vectors, matrices, and tensors.
- Always check **shape** and **ndim** before operations; most bugs in ML code are **dimension/axis** mistakes.
- **Vectorization** (operate on whole arrays) replaces Python loops and is the reason NumPy code is **fast**.
- **Broadcasting** lets different shapes work together (e.g., add a bias vector to every row of a batch) without manual copying.
- Learn the "daily tools": **indexing/slicing**, **reshape/flatten**, **axis** reductions (sum/mean), and **matrix multiplication** (`@`).
- Strong NumPy fundamentals make **Pandas**, **Scikit-Learn**, and **TensorFlow/PyTorch** much easier.

Key Takeaways

- **ndarray** is NumPy's main object: a fast, contiguous **n-dimensional array** used to store vectors, matrices, and tensors.
- Always check **shape** and **ndim** before operations; most bugs in ML code are **dimension/axis** mistakes.
- **Vectorization** (operate on whole arrays) replaces Python loops and is the reason NumPy code is **fast**.
- **Broadcasting** lets different shapes work together (e.g., add a bias vector to every row of a batch) without manual copying.
- Learn the "daily tools": **indexing/slicing**, **reshape/flatten**, **axis** reductions (sum/mean), and **matrix multiplication** (`@`).
- Strong NumPy fundamentals make **Pandas**, **Scikit-Learn**, and **TensorFlow/PyTorch** much easier.

Next Step: NumPy → Pandas → Scikit-Learn → Deep Learning

Pandas: Working with Tabular Data

Why Pandas?

Pandas is Python's standard library for working with **tabular data** (rows & columns).

Why Pandas?

Pandas is Python's standard library for working with **tabular data** (rows & columns).

- Built on top of **NumPy** $\Rightarrow$ fast operations on large datasets.

Why Pandas?

Pandas is Python's standard library for working with **tabular data** (rows & columns).

- Built on top of **NumPy** $\Rightarrow$ fast operations on large datasets.
- The **DataFrame** is like an Excel table with **column names**, **row indices**, and mixed data types.

Why Pandas?

Pandas is Python's standard library for working with **tabular data** (rows & columns).

- Built on top of **NumPy** $\Rightarrow$ fast operations on large datasets.
- The **DataFrame** is like an Excel table with **column names**, **row indices**, and mixed data types.
- Makes **data cleaning** easy: handle missing values, fix types, remove duplicates.

Why Pandas?

Pandas is Python's standard library for working with **tabular data** (rows & columns).

- Built on top of **NumPy** $\Rightarrow$ fast operations on large datasets.
- The **DataFrame** is like an Excel table with **column names**, **row indices**, and mixed data types.
- Makes **data cleaning** easy: handle missing values, fix types, remove duplicates.
- Supports **filtering**, **groupby/aggregation**, and **feature engineering** in a few lines.

Why Pandas?

Pandas is Python's standard library for working with **tabular data** (rows & columns).

- Built on top of **NumPy** $\Rightarrow$ fast operations on large datasets.
- The **DataFrame** is like an Excel table with **column names**, **row indices**, and mixed data types.
- Makes **data cleaning** easy: handle missing values, fix types, remove duplicates.
- Supports **filtering**, **groupby/aggregation**, and **feature engineering** in a few lines.
- Bridges to ML tools: select features, then export to **NumPy arrays** for models.

Why Pandas?

Pandas is Python's standard library for working with **tabular data** (rows & columns).

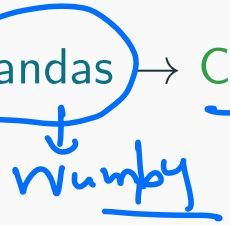
- Built on top of **NumPy** $\Rightarrow$ fast operations on large datasets.
- The **DataFrame** is like an Excel table with **column names**, **row indices**, and mixed data types.
- Makes **data cleaning** easy: handle missing values, fix types, remove duplicates.
- Supports **filtering**, **groupby/aggregation**, and **feature engineering** in a few lines.
- Bridges to ML tools: select features, then export to **NumPy arrays** for models.

Why Pandas?

Pandas is Python's standard library for working with **tabular data** (rows & columns).

- Built on top of **NumPy** $\Rightarrow$ fast operations on large datasets.
- The **DataFrame** is like an Excel table with **column names**, **row indices**, and mixed data types.
- Makes **data cleaning** easy: handle missing values, fix types, remove duplicates.
- Supports **filtering**, **groupby/aggregation**, and **feature engineering** in a few lines.
- Bridges to ML tools: select features, then export to **NumPy arrays** for models.

Typical workflow: Raw data $\rightarrow$ Pandas $\rightarrow$ Clean & explore $\rightarrow$ Features $\rightarrow$ Modeling



Core Data Structures

- **Series**: 1-dimensional labeled array
- **DataFrame**: 2-dimensional table with rows and columns (most used)

```
import pandas as pd

s = pd.Series([10, 20, 30, 40])
print(s)
```

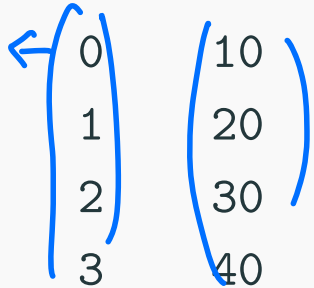
Core Data Structures

- **Series**: 1-dimensional labeled array
- **DataFrame**: 2-dimensional table with rows and columns (most used)

```
import pandas as pd

s = pd.Series([10, 20, 30, 40])
print(s)
```

Output:



```
0    10
1    20
2    30
3    40
dtype: int64
```

Creating a DataFrame

```
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [22, 24, 23],
    "Score": [90, 85, 95]
}

df = pd.DataFrame(data)
print(df)
```

Creating a DataFrame

```
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [22, 24, 23],
    "Score": [90, 85, 95]
}

df = pd.DataFrame(data)
print(df)
```

Output:

```
      Name  Age  Score
0    Alice   22     90
1      Bob   24     85
2  Charlie   23     95
```

Reading Data Files

```
df = pd.read_csv("students.csv") # df = pd.read_excel("students.xlsx")
print(df.head())
print(df.shape)
print(df.info())
```

Reading Data Files

```
df = pd.read_csv("students.csv") # df = pd.read_excel("students.xlsx")
print(df.head())
print(df.shape)
print(df.info())
```

Common Output:

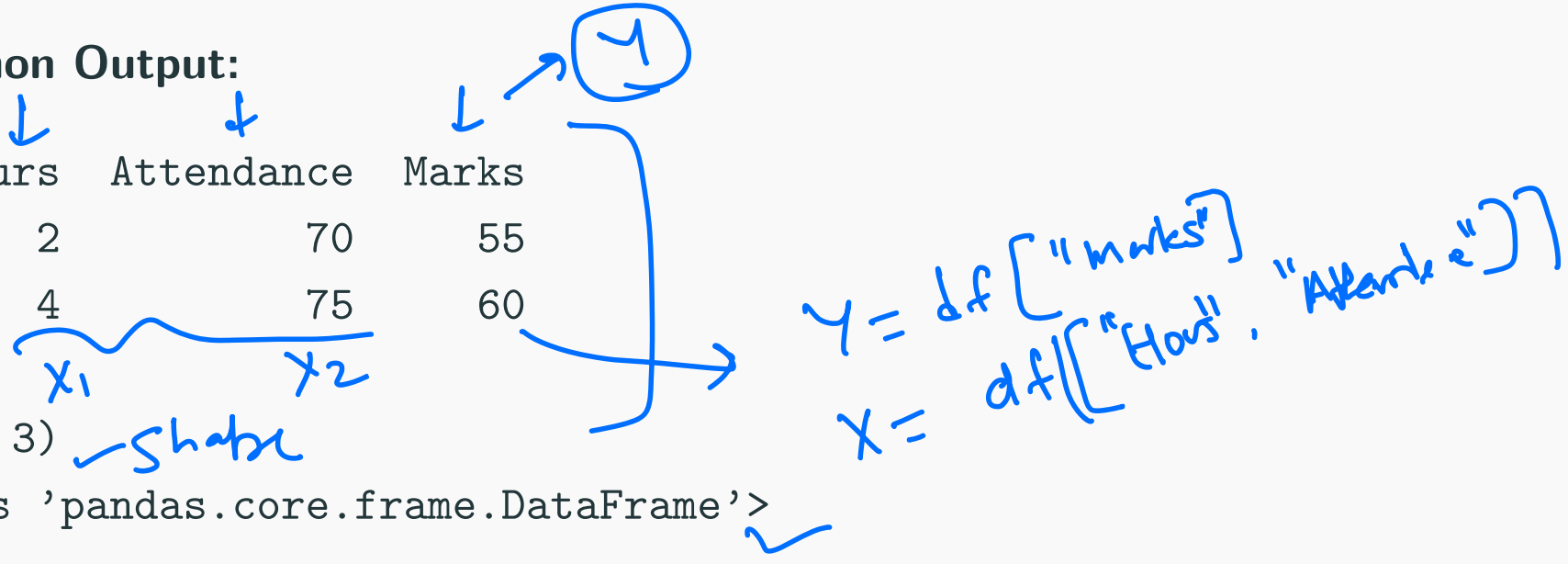
```
   Hours  Attendance  Marks
0      2          70     55
1      4          75     60
...
(100, 3)
<class 'pandas.core.frame.DataFrame'>
...
```

Reading Data Files

```
df = pd.read_csv("students.csv") # df = pd.read_excel("students.xlsx")
print(df.head())
print(df.shape)
print(df.info())
```

Common Output:

```
   Hours  Attendance  Marks
0      2          70     55
1      4          75     60
...
(100, 3)
<class 'pandas.core.frame.DataFrame'>
...
```



Note: Most machine learning projects start with CSV files.

Selecting Columns

```
print(df["Age"])
print(df[["Age", "Score"]])
```

Selecting Columns

```
print(df["Age"])
print(df[["Age", "Score"]])
```

Output (partial):

```
0    22
1    24
2    23
Name: Age, dtype: int64
```

```
   Age  Score
0   22     90
1   24     85
2   23     95
```

Selecting columns = Selecting features for ML models.

Row Selection with loc and iloc

```
print(df.loc[0])          # label-based
print(df.iloc[0:2])       # position-based
```

Output:

```
Name      Alice
Age          22
Score        90
Name: 0, dtype: object
```

```
    Name  Age  Score
0  Alice   22     90
1    Bob   24     85
```

Key Difference: `loc` uses labels, `iloc` uses integer positions.

Filtering Data

```
print(df[df["Age"] > 22])

print(df[(df["Age"] > 22) & (df["Score"] > 85)])
```

Output (example):

```
      Name  Age  Score
1      Bob   24     85
2  Charlie   23     95

      Name  Age  Score
2  Charlie   23     95
```

Useful for selecting specific observations before training.

Adding New Columns & Feature Engineering

```
df["Bonus"] = 5
df["FinalScore"] = df["Score"] + df["Bonus"]

print(df[["Score", "Bonus", "FinalScore"]])
```

Output:

	Score	Bonus	FinalScore
0	90	5	95
1	85	5	90
2	95	5	100

Feature engineering is a key step in improving model performance.

Descriptive Statistics

```
print(df.describe())

print("Mean Age:", df["Age"].mean())
print("Max Score:", df["Score"].max())
```

Output (partial):

```
              Age      Score
count    3.000000   3.000000
mean    23.000000  90.000000
std      1.000000   5.000000
...
```

Essential for understanding your dataset quickly.

Handling Missing Values

```
print(df.isnull().sum())

df_clean = df.dropna()                    # Remove
# df.fillna(df.mean(), inplace=True)      # Fill
```

Output:

```
Name     0
Age      0
Score    0
dtype: int64
```

Data cleaning is often the most time-consuming part of ML projects.

Combining Pandas with NumPy

```python
import pandas as pd
import numpy as np

# Assume df is already a pandas DataFrame
X = df[["Age", "Score"]].values   # Convert to NumPy array
print(type(X))
print(X.shape)
```

Output:

```
<class 'numpy.ndarray'>
(3, 2)
```

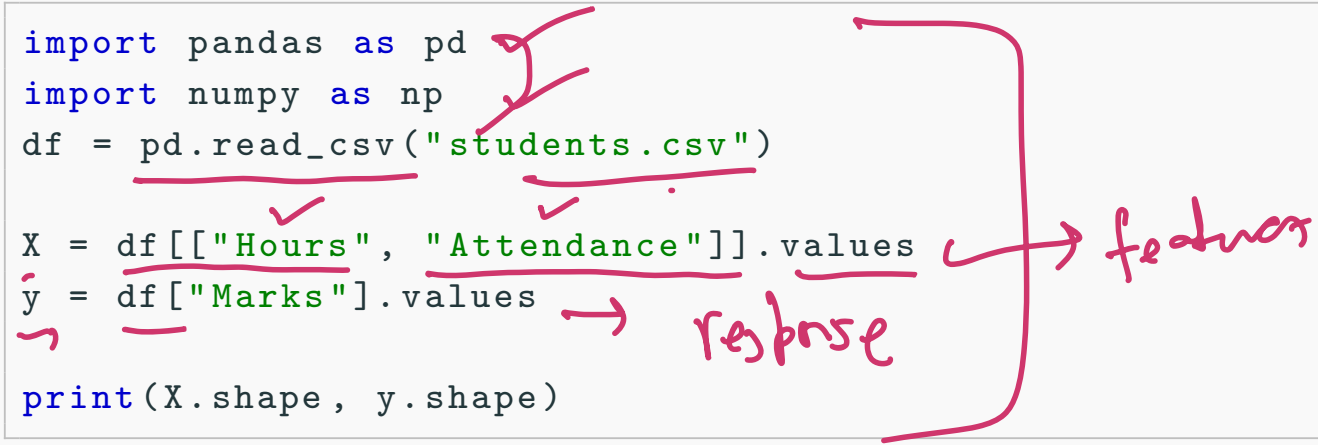
This NumPy matrix is what gets fed into machine learning models.

Mini Machine Learning Workflow

```
import pandas as pd
import numpy as np
df = pd.read_csv("students.csv")

X = df[["Hours", "Attendance"]].values
y = df["Marks"].values

print(X.shape, y.shape)
```



Mini Machine Learning Workflow

```
import pandas as pd
import numpy as np
df = pd.read_csv("students.csv")

X = df[["Hours", "Attendance"]].values
y = df["Marks"].values

print(X.shape, y.shape)
```

Pipeline Summary:

1. Load data with Pandas ✓

Mini Machine Learning Workflow

```
import pandas as pd
import numpy as np
df = pd.read_csv("students.csv")

X = df[["Hours", "Attendance"]].values
y = df["Marks"].values

print(X.shape, y.shape)
```

Pipeline Summary:

1. Load data with Pandas
2. Clean & explore ✓

Mini Machine Learning Workflow

```
import pandas as pd
import numpy as np
df = pd.read_csv("students.csv")

X = df[["Hours", "Attendance"]].values
y = df["Marks"].values

print(X.shape, y.shape)
```

Pipeline Summary:

1. Load data with Pandas
2. Clean & explore
3. Select features

Mini Machine Learning Workflow

```
import pandas as pd
import numpy as np
df = pd.read_csv("students.csv")

X = df[["Hours", "Attendance"]].values
y = df["Marks"].values

print(X.shape, y.shape)
```

Pipeline Summary:

1. Load data with Pandas
2. Clean & explore
3. Select features
4. Convert to NumPy ✓

Mini Machine Learning Workflow

```
import pandas as pd
import numpy as np
df = pd.read_csv("students.csv")

X = df[["Hours", "Attendance"]].values
y = df["Marks"].values

print(X.shape, y.shape)
```

Pipeline Summary:

1. Load data with Pandas
2. Clean & explore
3. Select features
4. Convert to NumPy
5. Train model

Pandas Mini Exercise

Given:

```
df = pd.DataFrame({
    "Name": ["A", "B", "C"],
    "Age": [20, 25, 22],
    "Marks": [80, 95, 88]
})
```

Tasks (write the Pandas code):

1. Compute **mean age** (hint: `df["Age"].mean()`)

Solutions will be discussed in lab.

Pandas Mini Exercise

Given:

```
df = pd.DataFrame({
    "Name": ["A", "B", "C"],
    "Age": [20, 25, 22],
    "Marks": [80, 95, 88]
})
```

Tasks (write the Pandas code):

1. Compute **mean age** (hint: `df["Age"].mean()`)
2. **Filter** students with **Marks** $>$ **85** (hint: boolean mask)

Solutions will be discussed in lab.

Pandas Mini Exercise

Given:

```
df = pd.DataFrame({
    "Name": ["A", "B", "C"],
    "Age": [20, 25, 22],
    "Marks": [80, 95, 88]
})
```

Tasks (write the Pandas code):

1. Compute **mean age** (hint: `df["Age"].mean()`)
2. **Filter** students with **Marks** $>$ **85** (hint: boolean mask)
3. Add a **Bonus** column (+5) (hint: assignment)

Solutions will be discussed in lab.

Pandas Mini Exercise

Given:

```
df = pd.DataFrame({
    "Name": ["A", "B", "C"],
    "Age": [20, 25, 22],
    "Marks": [80, 95, 88]
})
```

Tasks (write the Pandas code):

1. Compute **mean age** (hint: `df["Age"].mean()`)
2. **Filter** students with **Marks** $>$ **85** (hint: boolean mask)
3. Add a **Bonus** column (+5) (hint: assignment)
4. **Sort** by Marks (descending) (hint: `sort_values`)

Solutions will be discussed in lab.

Pandas Mini Exercise

Given:

```
df = pd.DataFrame({
    "Name": ["A", "B", "C"],
    "Age": [20, 25, 22],
    "Marks": [80, 95, 88]
})
```

Tasks (write the Pandas code):

1. Compute **mean age** (hint: `df["Age"].mean()`)
2. **Filter** students with **Marks** $>$ **85** (hint: boolean mask)
3. Add a **Bonus** column (+5) (hint: assignment)
4. **Sort** by Marks (descending) (hint: `sort_values`)
5. Convert **Marks** to a **NumPy array** (hint: `to_numpy()`)

Solutions will be discussed in lab.

Pandas Key Takeaways

- **DataFrame** = the central object for **tabular data** (like a spreadsheet, but programmable).

Pandas Key Takeaways

- **DataFrame** = the central object for **tabular data** (like a spreadsheet, but programmable).
- Use **loc/iloc** for reliable row selection and **column selection** for features.

Pandas Key Takeaways

- **DataFrame** = the central object for **tabular data** (like a spreadsheet, but programmable).
- Use **loc/iloc** for reliable row selection and **column selection** for features.
- Most ML prep is **cleaning** + **feature engineering** (missing values, new columns, transforms).

Pandas Key Takeaways

- **DataFrame** = the central object for **tabular data** (like a spreadsheet, but programmable).
- Use **loc/iloc** for reliable row selection and **column selection** for features.
- Most ML prep is **cleaning** + **feature engineering** (missing values, new columns, transforms).
- Before modeling, convert to **NumPy**: `X = df[cols].to_numpy()` and `y = df[target].to_numpy()`.

Pandas Key Takeaways

- **DataFrame** = the central object for **tabular data** (like a spreadsheet, but programmable).
- Use **loc/iloc** for reliable row selection and **column selection** for features.
- Most ML prep is **cleaning** + **feature engineering** (missing values, new columns, transforms).
- Before modeling, convert to **NumPy**: `X = df[cols].to_numpy()` and `y = df[target].to_numpy()`.

Pandas Key Takeaways

- **DataFrame** = the central object for **tabular data** (like a spreadsheet, but programmable).
- Use **loc/iloc** for reliable row selection and **column selection** for features.
- Most ML prep is **cleaning** + **feature engineering** (missing values, new columns, transforms).
- Before modeling, convert to **NumPy**: `X = df[cols].to_numpy()` and `y = df[target].to_numpy()`.

NumPy + Pandas = Foundation of AI Data Processing

matplotlib and seaborn: Data Visualization & Preprocessing

Why Visualize First?

Data visualization turns raw numbers into **insights** you can act on.

Visualize → Understand → Clean → Engineer features → Model

Why Visualize First?

Data visualization turns raw numbers into **insights** you can act on.

Visualize → Understand → Clean → Engineer features → Model

Key benefits:

- Discover **patterns** and **relationships** (signal).

Why Visualize First?

Data visualization turns raw numbers into **insights** you can act on.

Visualize → Understand → Clean → Engineer features → Model

Key benefits:

- Discover **patterns** and **relationships** (signal).
- Spot **outliers** and **data quality** issues early.

Why Visualize First?

Data visualization turns raw numbers into **insights** you can act on.

Visualize → Understand → Clean → Engineer features → Model

Key benefits:

- Discover **patterns** and **relationships** (signal).
- Spot **outliers** and **data quality** issues early.
- Guide **feature engineering** decisions.

Why Visualize First?

Data visualization turns raw numbers into **insights** you can act on.

Visualize → Understand → Clean → Engineer features → Model

Key benefits:

- Discover **patterns** and **relationships** (signal).
- Spot **outliers** and **data quality** issues early.
- Guide **feature engineering** decisions.
- Build intuition before trying complex models (debug faster).

Visualization in the AI Workflow

Raw data → Cleaning → Visualization → Feature engineering → Modeling

Visualization helps you answer:

- Are variables **correlated** (useful signal)?

Visualization in the AI Workflow

Raw data → Cleaning → Visualization → Feature engineering → Modeling

Visualization helps you answer:

- Are variables **correlated** (useful signal)?
- Are there **outliers** or **skewed** distributions?

Visualization in the AI Workflow

Raw data → Cleaning → Visualization → Feature engineering → Modeling

Visualization helps you answer:

- Are variables **correlated** (useful signal)?
- Are there **outliers** or **skewed** distributions?
- Which **features** look promising before modeling?

Matplotlib Basics

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

plt.figure(figsize=(5,3))
plt.plot(x, y, marker='o')
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.title("Study Hours vs Performance")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

Matplotlib Basics

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

plt.figure(figsize=(5,3))
plt.plot(x, y, marker='o')
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.title("Study Hours vs Performance")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

What to look for: trend, outliers, and non-linearity.

Seaborn Basics

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

# Quick plot directly from a DataFrame
sns.scatterplot(data=df, x="Hours", y="Marks", hue="Attendance")
plt.title("Hours vs Marks (colored by Attendance)")
plt.show()
```

Seaborn Basics

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

# Quick plot directly from a DataFrame
sns.scatterplot(data=df, x="Hours", y="Marks", hue="Attendance")
plt.title("Hours vs Marks (colored by Attendance)")
plt.show()
```

Why Seaborn (vs Matplotlib)?

- **Cleaner defaults**: themes, grids, nicer colors.

Seaborn Basics

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

# Quick plot directly from a DataFrame
sns.scatterplot(data=df, x="Hours", y="Marks", hue="Attendance")
plt.title("Hours vs Marks (colored by Attendance)")
plt.show()
```

Why Seaborn (vs Matplotlib)?

- **Cleaner defaults**: themes, grids, nicer colors.
- **DataFrame-friendly**: works directly with `data=...`, column names, and grouping (`hue`).

Seaborn Basics

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

# Quick plot directly from a DataFrame
sns.scatterplot(data=df, x="Hours", y="Marks", hue="Attendance")
plt.title("Hours vs Marks (colored by Attendance)")
plt.show()
```

Why Seaborn (vs Matplotlib)?

- **Cleaner defaults**: themes, grids, nicer colors.
- **DataFrame-friendly**: works directly with `data=...`, column names, and grouping (`hue`).
- **Statistical plots** built-in: KDE, violin plots, regression plots.

Seaborn Basics

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

# Quick plot directly from a DataFrame
sns.scatterplot(data=df, x="Hours", y="Marks", hue="Attendance")
plt.title("Hours vs Marks (colored by Attendance)")
plt.show()
```

Why Seaborn (vs Matplotlib)?

- **Cleaner defaults**: themes, grids, nicer colors.
- **DataFrame-friendly**: works directly with `data=...`, column names, and grouping (`hue`).
- **Statistical plots** built-in: KDE, violin plots, regression plots.
- Still uses Matplotlib underneath $\Rightarrow$ you can customize everything with `plt`.

Common Plot Types

Plot type	When to use it	Python command
Scatter	Relationship between two numeric variables	`sns.scatterplot(...)`
Histogram	Shape of a distribution (skew, multi-modal)	`plt.hist(...)` `sns.kdeplot(...)`
Box plot	Outliers + spread (median, IQR)	`sns.boxplot(...)`
Bar plot	Compare categories / counts	`sns.barplot(...)`
Heatmap	Correlations across many features	`sns.heatmap(...)`

Note: `sns` is Seaborn (typically: `import seaborn as sns`).

Common Plot Types

Plot type	When to use it	Python command
Scatter	Relationship between two numeric variables	`sns.scatterplot(...)`
Histogram	Shape of a distribution (skew, multi-modal)	`plt.hist(...)` `sns.kdeplot(...)`
Box plot	Outliers + spread (median, IQR)	`sns.boxplot(...)`
Bar plot	Compare categories / counts	`sns.barplot(...)`
Heatmap	Correlations across many features	`sns.heatmap(...)`

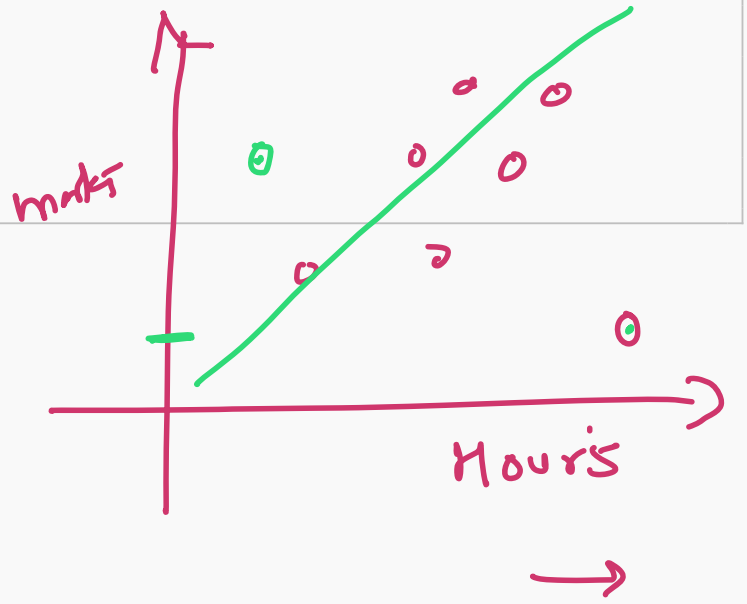
Note: `sns` is Seaborn (typically: `import seaborn as sns`).

Overall idea: One good plot that answers a question beats ten random plots.

Case Study 1: Scatter Plot (Relationships)

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.scatterplot(data=df, x="Hours", y="Marks")
plt.title("Hours Studied vs Marks")
plt.show()
```



Case Study 1: Scatter Plot (Relationships)

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.scatterplot(data=df, x="Hours", y="Marks")
plt.title("Hours Studied vs Marks")
plt.show()
```

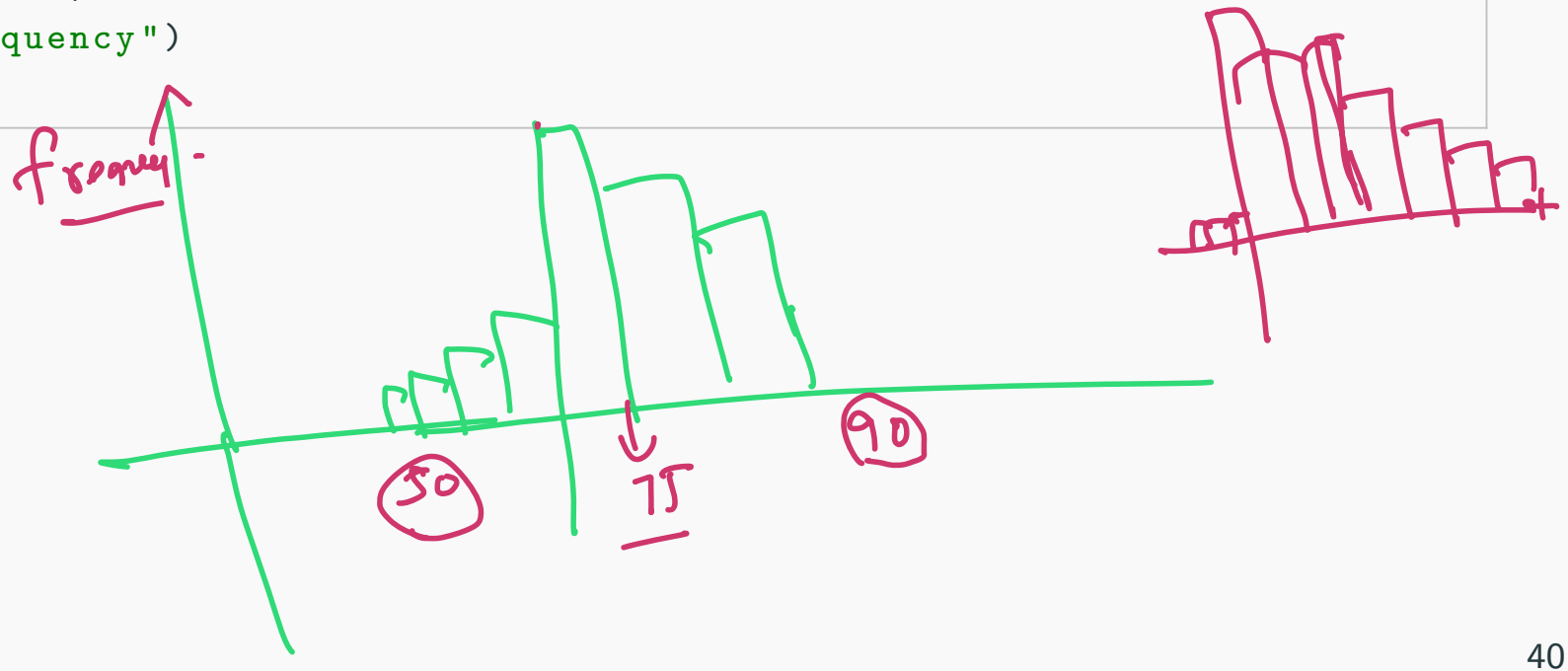
Interpretation:

- Look for **correlation** (upward/downward pattern).
- If the scatter is tight, **Hours** is likely a strong predictive feature.
- If it curves, you may need **non-linear** features or models.

Case Study 2: Histogram (Distributions)

```
import matplotlib.pyplot as plt

plt.hist(df["Marks"], bins=10, edgecolor='black')
plt.title("Distribution of Student Marks")
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.show()
```



Case Study 2: Histogram (Distributions)

```
import matplotlib.pyplot as plt

plt.hist(df["Marks"], bins=10, edgecolor='black')
plt.title("Distribution of Student Marks")
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.show()
```

Observations:

- Is it **skewed** (long tail) or roughly **normal**?

Case Study 2: Histogram (Distributions)

```
import matplotlib.pyplot as plt

plt.hist(df["Marks"], bins=10, edgecolor='black')
plt.title("Distribution of Student Marks")
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.show()
```

Observations:

- Is it **skewed** (long tail) or roughly **normal**?
- Are there **multiple peaks** (different groups in data)?

Case Study 2: Histogram (Distributions)

```
import matplotlib.pyplot as plt

plt.hist(df["Marks"], bins=10, edgecolor='black')
plt.title("Distribution of Student Marks")
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.show()
```

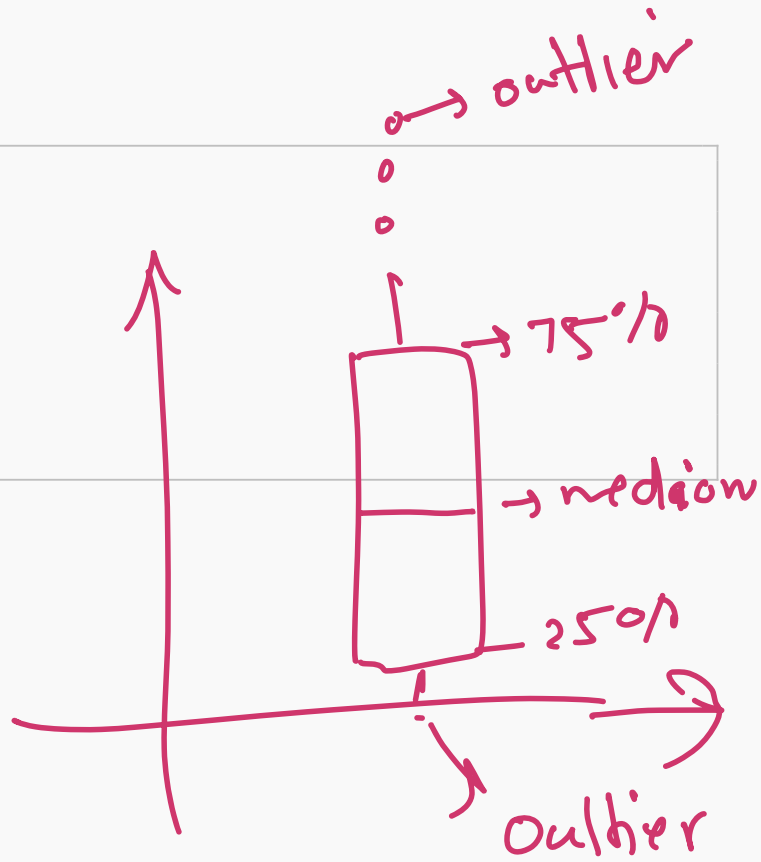
Observations:

- Is it **skewed** (long tail) or roughly **normal**?
- Are there **multiple peaks** (different groups in data)?
- Distribution shape guides **preprocessing** and model choice.

Case Study 3: Box Plot (Outliers)

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.boxplot(data=df, x="Marks")
plt.title("Marks Distribution (Outliers)")
plt.show()
```



Case Study 3: Box Plot (Outliers)

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.boxplot(data=df, x="Marks")
plt.title("Marks Distribution (Outliers)")
plt.show()
```

Note: Outliers can strongly affect training. Investigate them before removing.

Seaborn & Correlation Heatmap

```
import matplotlib.pyplot as plt
import seaborn as sns

corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Feature Correlation Matrix")
plt.show()
```

Seaborn & Correlation Heatmap

```
import matplotlib.pyplot as plt
import seaborn as sns

corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Feature Correlation Matrix")
plt.show()
```

Key insights:

- Spot **strong positive/negative correlations**.

Seaborn & Correlation Heatmap

```
import matplotlib.pyplot as plt
import seaborn as sns

corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Feature Correlation Matrix")
plt.show()
```

Key insights:

- Spot **strong positive/negative correlations**.
- Detect **redundant features** (very high correlation).

Seaborn & Correlation Heatmap

```
import matplotlib.pyplot as plt
import seaborn as sns

corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Feature Correlation Matrix")
plt.show()
```

Key insights:

- Spot **strong positive/negative correlations**.
- Detect **redundant features** (very high correlation).
- Identify likely **predictors** for the target variable.

Data Preprocessing Essentials

Real-world data is rarely **clean**. **Preprocessing** is crucial.

Data Preprocessing Essentials

Real-world data is rarely **clean**. **Preprocessing** is crucial.

Common tasks:

- Handle **missing values**
- Remove **duplicate records**
- **Scale features** (standardize/normalize)
- **Encode categorical** variables (make them numeric)
- Investigate **outliers** (fix, cap, or remove)

Data Preprocessing Essentials

Real-world data is rarely **clean**. **Preprocessing** is crucial.

Common tasks:

- Handle **missing values**
- Remove **duplicate records**
- **Scale features** (standardize/normalize)
- **Encode categorical** variables (make them numeric)
- Investigate **outliers** (fix, cap, or remove)

Rule of thumb: 60–80% of ML project time goes into data preparation.

Handling Missing Values

```
print(df.isnull().sum())

# Option 1: Remove rows with missing entries
df_clean = df.dropna()

# Option 2: Fill missing values (numeric columns)
# df.fillna(df.mean(numeric_only=True), inplace=True)
```

Handling Missing Values

```
print(df.isnull().sum())

# Option 1: Remove rows with missing entries
df_clean = df.dropna()

# Option 2: Fill missing values (numeric columns)
# df.fillna(df.mean(numeric_only=True), inplace=True)
```

Output (example):

```
Hours          0
Attendance     2
Marks          0
dtype: int64
```

Tip: Prefer fill when data is scarce; prefer drop when missingness is small.

Removing Duplicates

```
print("Duplicates:", df.duplicated().sum())

df = df.drop_duplicates()
```

Removing Duplicates

```
print("Duplicates:", df.duplicated().sum())

df = df.drop_duplicates()
```

Why it matters: Duplicate rows can bias training and inflate evaluation scores.

Feature Scaling

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

# Scale numeric feature columns (example)
df[["Hours", "Attendance"]] = scaler.fit_transform(df[["Hours", "Attendance"
    ]])
```

Feature Scaling

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

# Scale numeric feature columns (example)
df[["Hours", "Attendance"]] = scaler.fit_transform(df[["Hours", "Attendance"
    ]])
```

Why important? **Distance-based models** (KNN, SVM, K-means) are sensitive to feature scale. Without scaling, high-range features **dominate** the model.

Encoding Categorical Variables

```
import pandas as pd

# Convert categories to numeric columns (one-hot encoding)
df_encoded = pd.get_dummies(df, columns=["Gender"], drop_first=True)
print(df_encoded.head())
```

Encoding Categorical Variables

```python
import pandas as pd

# Convert categories to numeric columns (one-hot encoding)
df_encoded = pd.get_dummies(df, columns=["Gender"], drop_first=True)
print(df_encoded.head())
```

Output (example):

```
   Age  ...  Gender_Male
0   22  ...            1
1   24  ...            0
```

Why: Most ML models require **numerical inputs** (not raw text categories).

End-to-End Case Study (We will do in the lap part)

Student performance dataset (EDA → preprocessing → features)

```
import pandas as pd
import seaborn as sns

df = pd.read_csv("students.csv")
df = df.dropna()  # basic cleaning

# quick EDA
sns.heatmap(df.corr(numeric_only=True), annot=True)

# features + target
X = df[["Hours", "Attendance"]].to_numpy()
y = df["Marks"].to_numpy()
```

End-to-End Case Study (We will do in the lap part)

Student performance dataset (EDA → preprocessing → features)

```
import pandas as pd
import seaborn as sns

df = pd.read_csv("students.csv")
df = df.dropna()  # basic cleaning

# quick EDA
sns.heatmap(df.corr(numeric_only=True), annot=True)

# features + target
X = df[["Hours", "Attendance"]].to_numpy()
y = df["Marks"].to_numpy()
```

Result: clean, explored, and ready-to-train dataset.

Key Takeaways — EDA & Preprocessing

- **Visualize** first to find patterns, relationships, and problems early.
- Use **scatter**, **histograms**, and **heatmaps** as your core EDA toolkit.
- Good **preprocessing** (missing values, duplicates, scaling, encoding) improves model performance.
- **Clean** + **scaled** + **encoded** data is the foundation of successful AI.

Key Takeaways — EDA & Preprocessing

- **Visualize** first to find patterns, relationships, and problems early.
- Use **scatter**, **histograms**, and **heatmaps** as your core EDA toolkit.
- Good **preprocessing** (missing values, duplicates, scaling, encoding) improves model performance.
- **Clean** + **scaled** + **encoded** data is the foundation of successful AI.

Visualize → Clean → Train